

XIV Maratona UnB de Programação

25 de setembro de 2026

Ambiente de julgamento e sistema de submissão

Este documento descreve o ambiente que compila e executa as submissões de **XIV Maratona UnB de Programação**. Você pode editar este texto na aba  **Documentos** do painel do contest. O sistema preenche os blocos marcados.

1. Ambiente de julgamento

1.1 Ambiente

O sistema de julgamento roda em **Ubuntu 24.04.4 LTS x86_64**. As máquinas de julgamento reportam estes compiladores e interpretadores:

- C: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- C++: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Java: javac 21.0.11 / javac 21.0.12
- Kotlin: info: kotlinc-jvm 2.1.20 (JRE 21.0.11+10-1-24.04.2-Ubuntu) / info: kotlinc-jvm 2.1.20 (JRE 21.0.12+8-1-24.04-Ubuntu)
- Python 3 (PyPy3): Python 3.9.18 (7.3.15+dfsg-1build3, Apr 01 2024, 03:12:48)

1.2 Linguagens aceitas

O juiz aceita estas linguagens e extensões de arquivo:

Linguagens aceitas	Extensão do arquivo
C	.c
C++	.cpp, .cc, .cxx, .c++
Java	.java

Linguagens aceitas	Extensão do arquivo
--------------------	---------------------

Kotlin	.kt
--------	-----

Python 3 (PyPy3)	.py
------------------	-----

A extensão do arquivo que você envia identifica a linguagem. Envie o código-fonte com a extensão correta. O juiz rejeita um arquivo com extensão que não está na tabela. Renomear um arquivo não muda a linguagem: o servidor confere a linguagem de novo quando a submissão chega.

1.3 Limites de memória

Toda submissão, em toda linguagem, tem o limite total de memória de **1024 MB** e o tamanho máximo de pilha (*stack*) de **128 MB**. O juiz mede o pico de memória residente da submissão (a memória que o programa toca), e não o espaço de endereços que ele reserva.

Nota: o limite de memória inclui a memória do *runtime*. Em Java, Kotlin e Python o *runtime* usa vários megabytes. Em C e C++ o *runtime* é pequeno, mas não é zero.

1.4 Limites de tempo

Antes da prova, os juízes resolvem todos os problemas. O limite de tempo de cada problema vem do tempo de execução dessas soluções. A tabela vigente é:

Problema	Nome	C	C++	Java	Kotlin	Python
A	Árvore Six Seven	0.034	0.047	0.25	—	0.169
B	Bom Balanço de Divisores	0.04	0.12	0.34	—	0.12
C	Campinho	1.16	1.48	2.18	—	5.74
D	Desvendando as Formigas	0.42	1.336	1.101	—	2.135
E	Encontrando Ocorrências	0.6	0.14	1.92	—	2.12
F	Fada Quadrada	0.12	0.12	1.12	—	0.67

Problema	Nome	C	C++	Java	Kotlin	Python
G	Grande Desastre	—	—	—	—	—
H	Hugo e o Jogo das Palavras	0.034	0.034	0.29	—	0.155
I	Igualzim, não!	0.22	0.24	2.4	—	1.72
J	Jogo da Memória	4	4	4	4	4
K	Kolisão	—	0.26	0.7	—	0.66
L	Longe de Casa	1.4	2.5	3.66	—	4.16
M	Malha de Amplificadores	2.02	5.12	4.32	—	13.84
N	Nome de Usuário	0.07	0.07	0.92	—	0.27

* O limite de tempo depende da linguagem: uma coluna por linguagem.

O limite de tempo de cada problema também está disponível na interface do MOJ.

1.5 Outros limites

- Tamanho do arquivo-fonte: 1024 KB.
- Tamanho da saída do programa, incluindo a saída de erro padrão: 250 MB.
- Tempo de compilação: 30 segundos.

1.6 Compilação e execução

Compile e execute as suas soluções com os comandos das seções abaixo. Estes comandos não aplicam os limites de memória e de tempo do juiz, mas reproduzem o ambiente do juiz.

C e C++

- O juiz executa o arquivo executável que o compilador produz.
- O seu programa deve terminar com código de saída zero: termine o `main` com `return 0`, ou chame `exit(0)`.

- Em problemas com entradas grandes, `cin` e `cout` podem ser lentos. Por padrão eles usam um *buffer* sincronizado com a biblioteca `stdio`. Para usar `cin` e `cout`, chame `std::ios::sync_with_stdio(false);` no início do `main`. Nesse caso, não misture `scanf` e `printf` no mesmo programa.

- Compile soluções em C com:

```
gcc -lm -O2 -static {arquivo_enviado} -o main
```

- Compile soluções em C++ com:

```
g++ -lm -O2 -static -std=gnu++20 -pipe {arquivo_enviado} -o main
```

- Execute soluções em C e C++ com:

```
./main
```

Java

- O juiz executa a classe compilada que contém o `main`.
- Não declare `package` no seu programa. Um programa com *package* não roda no MOJ.
- O nome do arquivo deve ser igual ao da classe pública. Se o seu arquivo é `Klass.java`, a classe pública com o `main` deve ser `Klass`. Caso contrário você pode receber `Compilation Error`.
- Em problemas com entradas grandes, `Scanner` pode ser lento. Use E/S com *buffer*, por exemplo `BufferedReader` e `PrintWriter`.
- Compile soluções em Java com:

```
javac {arquivo_enviado}
```

- Execute soluções em Java com:

```
java -Xms10m -Xmx1024m -Xss131072k {nome_da_classe}
```

- O juiz define os tamanhos de *heap* e de pilha acima a partir dos limites de memória da seção 1.3. #### Kotlin

- O juiz executa o programa compilado.
- Não declare `package` no seu programa. Um programa com *package* não roda no MOJ.
- Não há regra para o nome do arquivo. O ponto de entrada é o fun `main()` de nível superior do arquivo que você envia.
- Compile soluções em Kotlin com:

```
kotlinc {arquivo_enviado} -include-runtime -d prog.jar
```

- Execute soluções em Kotlin com:

```
java -Xms10m -Xmx1024m -Xss131072k -jar prog.jar
```

- O juiz define os tamanhos de *heap* e de pilha acima a partir dos limites de memória da seção 1.3. ##### Python
- O juiz executa o arquivo que você envia com o interpretador **PyPy3**, e não com o CPython. O PyPy3 costuma ser mais rápido, mas alguns comportamentos são diferentes.
- O juiz confere a sintaxe do arquivo quando você o envia. Um arquivo com erro de sintaxe recebe Compilation Error.
- Confira a sintaxe de soluções em Python com:

```
pypy3 -m py_compile {arquivo_enviado}
```

- Execute soluções em Python com:

```
pypy3 {arquivo_enviado}
```

Para as outras linguagens aceitas, o juiz compila e executa o arquivo com as ferramentas padrão da linguagem, com os mesmos limites.

2. Como usar o sistema MOJ

2.1 Veredictos

Você pode receber estes veredictos:

- Not Answered Yet
- Accepted
- Wrong Answer
- Time Limit Exceeded
- Memory Limit Exceeded
- Runtime Error
- Compilation Error

Em uma prova o veredicto é tudo o que você recebe: nenhum caso de teste que falhou, nenhum *diff* e nenhuma pontuação parcial. Os dados de teste são material da prova.

Se você acha que um veredicto está errado, use a aba **Clarification**. Identifique a submissão pelo problema e pelo horário.

2.2 Notas de julgamento

- O juiz pode executar o seu programa em vários arquivos de entrada. Se o seu programa tem mais de um erro, por exemplo Time Limit Exceeded e Wrong Answer, você pode receber qualquer um deles.
- Se a compilação demora demais, ou se o programa compilado é grande demais, você recebe Compilation Error.
- Não existe Presentation Error nem Format Error. Se o problema pede a palavra *impossible* e você a escreve com um erro de digitação, o veredicto é Wrong Answer.
- Siga o formato da saída de exemplo. Espaços em branco extras dentro do razoável são aceitos: um espaço no fim de uma linha, um espaço entre valores, ou uma linha em branco no fim. Milhares de espaços em branco não são aceitos.
- Em problemas com saída em ponto flutuante, o juiz aceita qualquer resposta dentro da tolerância do enunciado. A tolerância é absoluta ou relativa, conforme o enunciado. A sua resposta não precisa ter os mesmos dígitos do exemplo.
- Quando o enunciado diz que várias saídas são aceitas, o juiz confere o formato e as restrições. O juiz não compara a sua saída com a saída de exemplo.
- Se o seu programa escreve saída demais, o veredicto é Runtime Error. A saída de erro padrão conta. Remova as mensagens de depuração antes de enviar.

2.3 Penalidades

Cada submissão que recebe um veredicto diferente de Accepted, antes do primeiro Accepted do mesmo problema, soma uma penalidade de **20 minutos** ao tempo total do time. Não há

penalidade para problemas que o time não resolve.

Veredictos sem penalidade: **Compilation Error**.

2.4 Tempos de resposta

O tempo até o veredicto depende do problema, do veredicto e do momento da prova. O juiz executa as submissões em paralelo, então você pode receber os veredictos fora de ordem. Algumas submissões precisam de conferência manual dos juízes. Nesse caso, uma demora de alguns minutos é normal.

2.5 Entrada e saída

1. Leia a entrada da **entrada padrão**.
2. Escreva a saída na **saída padrão**.
3. A entrada tem um caso de teste e não tem dados extras.
4. Quando uma linha tem vários valores, um espaço separa os valores.

Para mais dúvidas sobre o sistema de julgamento, leia a documentação do competidor na interface web.